

Mid-Period Progress Report: Cross-Rollup Arbitrage Detection and Simulation on Decentralized Exchanges

March 27, 2026

Abstract

This report presents the current research progress on cross-rollup arbitrage detection and simulation across decentralized exchanges, with primary coverage of Arbitrum, Base, and Optimism. The implemented system consists of two complementary components: (i) a single-chain detection pipeline that scans swap activity and reconstructs cyclic arbitrage transactions at the transaction level, and (ii) a simulation pipeline that enumerates reserve-token cycles, replays them under updated automated market maker (AMM) pool states, and estimates theoretical profitability under a deterministic synchronization rule.

The verified data stack is hybrid. The pool universe is seeded from Dune; reserve-token prices, initial pool states, and replay event histories are obtained from Allium; local MongoDB stores normalized pool updates and exported opportunities; and direct RPC access is used where on-chain state or event logs must be queried. The protocol mechanics currently implemented are Uniswap v2 and Uniswap v3, while the profitable simulation exports presently available are entirely Uniswap v3-based. The path-construction engine supports single-chain reserve-token cycles and pairwise cross-rollup spatial cycles of length up to five.

The current committed output artifacts are dated November 1, 2024 (UTC). On those stored exports, cross-chain simulation produces 43,199 windows and \$1.764M of total theoretical profit, while single-chain simulation produces 122,401 windows and \$890.98 of total theoretical profit. Realized single-chain detection outputs are far smaller in magnitude but economically more credible because they incorporate transaction costs: Base contributes the largest total detected profit (\$27,610.34), Arbitrum contributes \$1,297.31, and Optimism contributes \$113.08. These findings are informative, but they should be interpreted as research-stage results rather than deployable cross-rollup performance, because the present simulation does not yet model bridge settlement delay, inventory constraints, or adversarial competition.

1 Introduction

Liquidity fragmentation across Ethereum rollups can create temporary price discrepancies even when the assets are economically equivalent. In the current project, that problem is studied over Arbitrum, Optimism, and Base, with AMM mechanics drawn from Uniswap v2 and Uniswap v3. The economically relevant friction is not limited to AMM slippage; on OP Stack-style systems, transaction cost itself is two-part, combining L2 execution fees with an L1 data-availability component. Any serious empirical claim about arbitrage therefore must distinguish between *path profitability under synchronized states* and *profitability under executable cross-domain settlement*.

The current research is organized around four questions:

1. What data sources, chains, pool types, and reserve assets are currently active in the project?
2. How do the detection and simulation algorithms operate?
3. What do the currently stored empirical outputs show?
4. Which findings are already supported by implemented results, and which remain part of future work?

The present system should therefore be understood as a research infrastructure for opportunity generation, filtering, and replay, rather than as a production deployment stack.

2 System Scope

The implemented framework operates entirely off-chain as a research and simulation environment. Its purpose is to identify candidate opportunities, reconstruct realized single-chain arbitrage transactions, and replay reserve-token cycles under evolving pool states. The project does not yet include a bridge-aware cross-rollup executor or a smart-contract settlement layer.

2.1 Pool universe and reserve-token mapping

The pool universe is seeded from Dune and then filtered to retain Uniswap pools with valid fee metadata and unique pool addresses. The reserve-token seed set is

$$\mathcal{R} = \{\text{WETH}, \text{USDC}, \text{USDT}, \text{WBTC}, \text{DAI}\}.$$

Applying the current filtering logic to the committed pool universe yields:

- 57 unique Arbitrum pools: 55 Uniswap v3 and 2 Uniswap v2;
- 83 unique Base pools: 68 Uniswap v3 and 15 Uniswap v2;
- 17 unique Optimism pools: all 17 are Uniswap v3.

The reserve-token address mapping is not uniform across chains. WETH, USDC, and USDT exist on all three rollups; DAI exists on Arbitrum and Optimism only; and WBTC appears only on Arbitrum in the committed pool universe. As a consequence, not every reserve asset can anchor a cross-rollup mapping.

2.2 Verified data and infrastructure sources

The current project combines off-chain indexers, local persistence, and direct chain access. Table ?? summarizes the verified data stack.

Table 1: Verified data and infrastructure sources.

Component	Verified source	Technical role
Pool universe	Dune query API	Seeds the pool universe used in downstream filtering and path construction.
Reserve-token prices	Allium Explorer	Provides reserve-token price time series used for valuation and profit translation.
Initial v2 state	Allium Explorer	Provides initial reserve balances for constant-product pools.
Initial v3 state	Allium Explorer	Provides historical swap, mint, and burn information needed to initialize tick, liquidity, and square-root price state.
Replay event history (v2)	Allium Explorer	Provides replayable event histories for Uniswap v2 pools.
Replay event history (v3)	Allium Explorer	Provides replayable event histories for Uniswap v3 pools.
Tick initialization	Allium Explorer	Reconstructs tick-liquidity net maps from mint and burn histories.
Fallback v3 state queries	RPC endpoints	Retrieves <code>slot0()</code> and <code>liquidity()</code> when state must be refreshed at the relevant block.
Single-chain detection logs	Chain RPC providers	Retrieves swap logs and transaction receipts for transaction-level arbitrage reconstruction.
Persistence and replay store	Local MongoDB	Stores normalized pool updates and exported opportunity collections.

This source split matters for interpretation. The simulation pipeline is primarily driven by indexed historical data replayed through MongoDB, whereas the detection pipeline reconstructs realized arbitrage from observed on-chain transaction activity.

3 Data Methodology

3.1 Covered rollups and protocols

The currently implemented rollups are:

- Arbitrum,
- Base,

- Optimism.

The AMM protocols currently modeled are:

- Uniswap v2,
- Uniswap v3.

Although the broader research motivation includes cross-rollup DEX arbitrage more generally, the currently stored empirical outputs are drawn from the Uniswap-based subset of that design.

3.2 State initialization and temporal alignment

The replay engine initializes each pool from stored initial state. For Uniswap v2 pools, the required state variables are token reserves. For Uniswap v3 pools, the replay requires current tick, square-root price, active liquidity, and a tick-liquidity net map.

Reserve-token prices are aligned by nearest previous timestamp:

$$p_{c,a}(t) = p_{c,a}(s^*), \quad s^* = \max\{s \leq t : s \in \mathcal{T}_{c,a}\},$$

where c denotes chain, a token address, and $\mathcal{T}_{c,a}$ is the observed timestamp set for that token-price series. This is a backward-fill rule rather than interpolation.

4 Single-Chain Detection Methodology

4.1 Transaction-level arbitrage reconstruction

The single-chain detector scans swap logs for Uniswap v2 and Uniswap v3 swap events, groups them by block number and transaction index, orders them by log index, and tests whether the resulting sequence forms a valid cyclic token path. This is not a triangle-only detector. In the currently stored outputs, realized detected paths span:

- 2 to 6 swaps on Arbitrum;
- 2 to 7 swaps on Base;
- 2 to 5 swaps on Optimism.

For a candidate sequence $S = (s_1, \dots, s_m)$, the continuity checks are

$$\text{token}_k^{\text{out}} = \text{token}_{k+1}^{\text{in}}, \tag{1}$$

$$\text{amt}_k^{\text{out}} \geq \text{amt}_{k+1}^{\text{in}}, \tag{2}$$

$$\text{pool}_k \neq \text{pool}_{k+1}, \tag{3}$$

for each adjacent pair (s_k, s_{k+1}) . A cycle closes when the first input token equals the final output token, with ETH and WETH treated as equivalent wrappers. The first input amount must also be no larger than the closing output amount.

Accordingly, the detector is best described as *transaction-level cyclic swap reconstruction under token continuity constraints*.

4.2 Gain, cost, and net profit

For a detected sequence, the token balance for asset a is

$$B_a = \sum_{j=1}^m \text{out}_{j,a} - \sum_{j=1}^m \text{in}_{j,a}.$$

Using token decimals d_a and price lookup $p_a(t)$ at block timestamp t , the detector computes

$$G(t) = \sum_{a:B_a>0} \frac{B_a}{10^{d_a}} p_a(t), \quad C_{\text{token}}(t) = \sum_{a:B_a<0} \frac{-B_a}{10^{d_a}} p_a(t).$$

Net profit is then

$$\Pi(t) = G(t) - C_{\text{token}}(t) - C_{\text{gas}}(t).$$

For Arbitrum, the gas term is computed from gas used and gas price. For Base and Optimism, the gas term includes both L2 execution and the L1 data-availability component where those receipt fields are available. As a consequence, the stored single-chain detection outputs already represent gas-adjusted realized profits rather than gross spread only.

5 Arbitrage Detection Algorithm for Replay Simulation

5.1 Reserve-token cycle construction

The path-construction engine builds a graph over AMM pools and reserve-token endpoints, with maximum path length

$$L_{\max} = 5.$$

Single-chain paths are reserve-token cycles that begin and end in the same reserve asset. Cross-rollup paths are pairwise concatenations of two chain-local reserve-token paths. If $M_{c_1 \rightarrow c_2}$ denotes the symbol-based address mapping from chain c_1 to chain c_2 , the builder seeks path pairs satisfying

$$M_{c_1 \rightarrow c_2}(\text{end}(\pi^{(1)})) = \text{start}(\pi^{(2)}), \quad M_{c_2 \rightarrow c_1}(\text{end}(\pi^{(2)})) = \text{start}(\pi^{(1)}),$$

with

$$|\pi^{(1)}| + |\pi^{(2)}| \leq 5.$$

This design implies that the implemented cross-rollup strategy class is spatial cross-rollup arbitrage via reserve-token mapping rather than a coded bridge-transfer protocol.

5.2 Gain screen and route valuation

Before exact replay, the engine applies a route-level log-gain screen. For a Uniswap v2 route, the fee-adjusted directional price is

$$P^{(v2)} = \begin{cases} \frac{R_{\text{out}}}{R_{\text{in}}} 10^{d_{\text{in}} - d_{\text{out}}} (1 - 0.003), & \text{if zero-for-one,} \\ \frac{R_{\text{in}}}{R_{\text{out}}} 10^{d_{\text{out}} - d_{\text{in}}} (1 - 0.003), & \text{otherwise.} \end{cases}$$

For a Uniswap v3 route, the code reconstructs the implied tick from the stored Q64.96 square-root price:

$$\tau = \log_{1.0001} \left(\frac{\text{sqrt_price}^2}{2^{192}} \right).$$

The directional price is then

$$P^{(v3)} = \begin{cases} 1.0001^\tau 10^{d_{\text{in}} - d_{\text{out}}} (1 - f), & \text{if zero-for-one,} \\ \left(1.0001^\tau 10^{d_{\text{out}} - d_{\text{in}}} \right)^{-1} (1 - f), & \text{otherwise,} \end{cases}$$

where f is the fee tier in decimal form.

Each route contributes

$$g_k = -\log_2 P_k, \quad G(\pi) = \sum_k g_k,$$

and a path is retained only if

$$-1 < G(\pi) < 0.$$

The engine also computes a liquidity key per route. For v2 it uses $\sqrt{R_{\text{in}} R_{\text{out}}}$; for v3 it uses active liquidity. The path is ranked using the minimum normalized liquidity across its routes. This should be interpreted as a coarse prioritization heuristic rather than a dimensionally exact liquidity measure.

5.3 Exact replay and slippage model

If every route in a path is Uniswap v2 and the total path length is 2, 3, 4, or 5, the engine applies closed-form optimization routines. The underlying swap function is

$$\Delta y = R_{\text{out}} - \frac{R_{\text{in}} R_{\text{out}}}{R_{\text{in}} + 0.997 \Delta x} = \frac{0.997 \Delta x R_{\text{out}}}{R_{\text{in}} + 0.997 \Delta x}.$$

Otherwise, the engine optimizes input size numerically through ternary search:

$$x^* \in \arg \max_{x \geq 0} \Pi_\pi(x).$$

For Uniswap v3, replay steps across ticks. Let P denote normalized square-root price and L active

liquidity. The token deltas within a tick range are

$$\Delta x = L \left(\frac{1}{P_b} - \frac{1}{P_a} \right), \quad \Delta y = L(P_b - P_a),$$

with sign determined by swap direction. The next square-root price is updated as

$$P' = \begin{cases} \frac{LP}{L + \Delta x P}, & \text{zero-for-one,} \\ P + \frac{\Delta x}{L}, & \text{one-for-zero.} \end{cases}$$

The input amount is fee-adjusted by $(1 - f)$ prior to the state transition.

6 Simulation Framework

6.1 Conflict elimination and synchronization semantics

After replay, profitable paths are sorted by profit and greedily deconflicted: if a lower-ranked path shares any pool address with a higher-ranked one, it is discarded. Therefore the exported total profit per window is a sum of non-conflicting path profits rather than the sum over every individually positive path.

The framework supports two replay modes:

- **Single-chain mode:** iterates second by second, groups updates by block number within each second, and replays per updated block.
- **Cross-chain mode:** iterates over a user-specified time window and replays all paths touched by updates whose timestamps fall in that interval.

The currently stored cross-chain exports all satisfy

$$\text{timestamp_range_stop} - \text{timestamp_range_start} = 2 \text{ seconds.}$$

Hence the stored cross-rollup study is a 2-second wall-clock synchronization experiment rather than a bridge-aware latency model.

In cross-chain mode, pool updates are deduplicated by pool address within each window. The retained record serves as a representative per-pool state for that window, which is appropriate for deterministic replay but not equivalent to executable cross-domain settlement.

6.2 Established progress

The following progress has already been achieved:

1. multi-rollup data coverage for Arbitrum, Base, and Optimism;
2. protocol-specific replay for both Uniswap v2 and Uniswap v3 mechanics;

3. reserve-token path generation for both single-chain and pairwise cross-rollup cycles;
4. incremental state updating driven by observed pool changes;
5. gain-based filtering and exact replay for selected candidates;
6. conflict-aware aggregation of profitable paths;
7. MongoDB-based persistence of replay and detection outputs.

Accordingly, the current state of the project is best described as a functioning research prototype for cross-rollup arbitrage generation and replay, with robust pool-side mechanics and an execution layer that remains incomplete for full deployment realism.

7 Preliminary Data Analysis

All currently stored result files correspond to November 1, 2024 (UTC). The earliest stored simulation timestamp is 1730419200, i.e., 2024-11-01 00:00:00 UTC, and the detection exports lie on the same date.

7.1 Aggregate metrics

Table ?? reports aggregate row-level outputs over the stored exports.

Table 2: Aggregate metrics over currently stored November 1, 2024 exports. Detection PnL rows count only rows with numeric profit fields.

Dataset	Files	Rows	PnL rows	Positive	Zero	Negative	Sum PnL (USD)	Mean PnL (USD)
Cross-chain simulation	5	43,199	43,199	34,886	8,313	0	1,764,433.18	40.8443
Single-chain simulation	13	122,401	122,401	13,716	108,685	0	890.98	0.007279
Detection: Arbitrum	1	5,409	5,235	4,176	0	1,059	1,297.31	0.2478
Detection: Base	3	22,860	21,905	17,090	0	4,815	27,610.34	1.2605
Detection: Optimism	1	7,890	7,846	7,741	0	105	113.08	0.0144

Three conclusions follow immediately.

1. Cross-chain replay dominates single-chain replay in the stored simulation outputs. The gap between \$1.764M and \$890.98 should not be interpreted naively as deployable superiority; it is more plausibly related to the permissive 2-second synchronization rule and the absence of bridge settlement frictions.
2. Realized single-chain detection is much smaller in scale but methodologically stronger, because detection rows incorporate transaction costs and can therefore be positive or negative.
3. Base detection is highly heavy-tailed. Its mean net profit is \$1.2605, but that average is influenced materially by a small number of very large outliers.

7.2 Simulation output structure

The profitable simulation exports are more specific than Table ?? alone suggests.

- All profitable simulation paths in the currently stored outputs are Uniswap v3-only.
- All profitable cross-rollup paths are pairwise rather than three-rollup opportunities.
- Path lengths are variable rather than restricted to triangular cycles.

In the stored outputs, profitable cross-rollup path lengths are distributed as follows:

$$2 : 10,819, \quad 3 : 25,963, \quad 4 : 12,560, \quad 5 : 8,354.$$

Profitable single-chain path lengths are distributed as

$$2 : 204, \quad 3 : 6,536, \quad 4 : 3,726, \quad 5 : 4,491.$$

At the opportunity level, the currently stored outputs contain:

- 57,696 profitable non-conflicting cross-rollup opportunities, with mean profit \$30.58, median profit \$0.5207, mean input notional \$49,984.15, and median input notional \$1,536.20;
- 14,957 profitable non-conflicting single-chain opportunities, with mean profit \$0.05957, median profit \$0.01492, mean input notional \$163.82, and median input notional \$75.40.

The cross-rollup distribution is therefore highly skewed. Arbitrum–Base opportunities sum to \$878,535.32, Base–Optimism opportunities sum to \$849,791.36, and Arbitrum–Optimism opportunities sum to \$36,106.49. Mean profit per profitable opportunity is highest on Base–Optimism (\$90.58), followed by Arbitrum–Base (\$21.89), with Arbitrum–Optimism materially smaller (\$4.42).

The replay engine is also selective. Averaged across stored windows:

- cross-chain replay touches 11,443.08 updated paths per window, keeps 354.86 positive-gain candidates, and simulates 29.73 paths exactly;
- single-chain replay touches 5,597.22 updated paths per window, keeps 115.10 positive-gain candidates, and simulates 2.38 paths exactly.

This large reduction indicates that most computational effort lies in state propagation and coarse filtering, not in exact replay.

7.3 Detection economics

Table ?? reports the detected single-chain economics using the stored gross gain, total cost, and net profit fields.

Table 3: Detected single-chain economics from currently stored outputs. Max drawdown is computed on cumulative net profit after sorting by block timestamp.

Chain	Mean gain	Mean cost	Mean net	Median net	Success rate	Max DD
Arbitrum	\$0.3831	\$0.1318	\$0.2478	\$0.00646	79.77%	\$38.34
Base	\$1.6867	\$0.4109	\$1.2605	\$0.00466	78.02%	\$7.02
Optimism	\$0.04006	\$0.02561	\$0.01441	\$0.00130	98.66%	\$0.16

This table sharpens the interpretation.

- Base has the largest realized outliers, but not the largest typical trade. Its mean net profit is highest, whereas the median detected net profit is only \$0.00466.
- Arbitrum is economically meaningful but less extreme. Its average detected profit is \$0.2478, with the largest cumulative drawdown among the three chains once trades are ordered through time.
- Optimism has the smallest absolute dollar profits. Its success rate is the highest, but both mean and median profits are small.

The protocol mix in detection is broader than in replay. The most common realized patterns are v3-to-v3 and v3-to-v3-to-v3, but mixed v2-to-v3 and v3-to-v2 sequences also occur on Arbitrum and Base. Thus the single-chain detection framework is genuinely mixed-protocol, even though the profitable replay outputs currently stored are v3-only.

7.4 Sharpe ratio and success-rate interpretation

A methodological caution is necessary when discussing Sharpe ratios. The current outputs do not define a capital time series for a deployable portfolio with compounding, financing, inventory management, bridge lock-up, or execution failure. Consequently, any Sharpe ratio computed directly from per-opportunity profit rows would be an opportunity-level descriptive statistic rather than an investable strategy Sharpe.

The appropriate descriptive object is therefore

$$S_{\text{opp}} = \frac{\mathbb{E}[\Pi]}{\sqrt{\text{Var}(\Pi)}},$$

where Π is per-opportunity net USD profit. This can be useful for comparing output dispersion across datasets, but it should not be annualized or presented as a portfolio Sharpe without further structure.

A more defensible measure at the current stage is the success rate

$$\text{SuccessRate} = \frac{N_+}{N_{\text{pnl}}},$$

where N_+ is the number of positive-profit observations and N_{pnl} is the number of observations with non-null profit.

Using the stored summary table, the implied success rates are approximately

$$\text{SuccessRate}_{\text{cross-chain sim}} \approx 80.8\%, \quad \text{SuccessRate}_{\text{single-chain sim}} \approx 11.2\%,$$

and for the detection outputs

$$\text{SuccessRate}_{\text{Arbitrum}} \approx 79.8\%, \quad \text{SuccessRate}_{\text{Base}} \approx 78.0\%, \quad \text{SuccessRate}_{\text{Optimism}} \approx 98.7\%.$$

These quantities are informative as descriptive statistics, but they should not be interpreted as direct estimates of deployable hit rate under a full execution model.

7.5 Impact of L1/L2 gas costs on net profit

The stored detection outputs explicitly decompose gross gain, total cost, and net profit. If gross economic edge is denoted by G and total execution cost by C , then

$$\Pi = G - C.$$

Hence

$$\frac{\partial \Pi}{\partial C} = -1.$$

The research implication is straightforward:

1. for small-edge opportunities, gas cost often determines the sign of net profit;
2. for larger opportunities, cost still materially compresses the realized edge;
3. the current cross-rollup simulation does not yet implement a full bridge-cost layer, so its reported profits should be interpreted as partial-cost or upper-bound estimates rather than final executable P&L.

7.6 Placeholder tables and figures for the final report

Table 4: Placeholder: Cross-Chain Simulation Metrics by Window Length

Window Length	Updated Pools	Simulated Paths	Positive Paths	Total Profit (USD)
1 second	—	—	—	—
2 seconds	—	—	—	—
5 seconds	—	—	—	—
10 seconds	—	—	—	—

Table 5: Placeholder: Profit Decomposition by Chain

Chain	Mean Gross Gain (USD)	Mean Cost (USD)	Mean Net Profit (USD)	Success Rate
Arbitrum	—	—	—	—
Base	—	—	—	—
Optimism	—	—	—	—



Figure 1: Placeholder: Distribution of Net Profit per Opportunity (single-chain versus cross-chain).

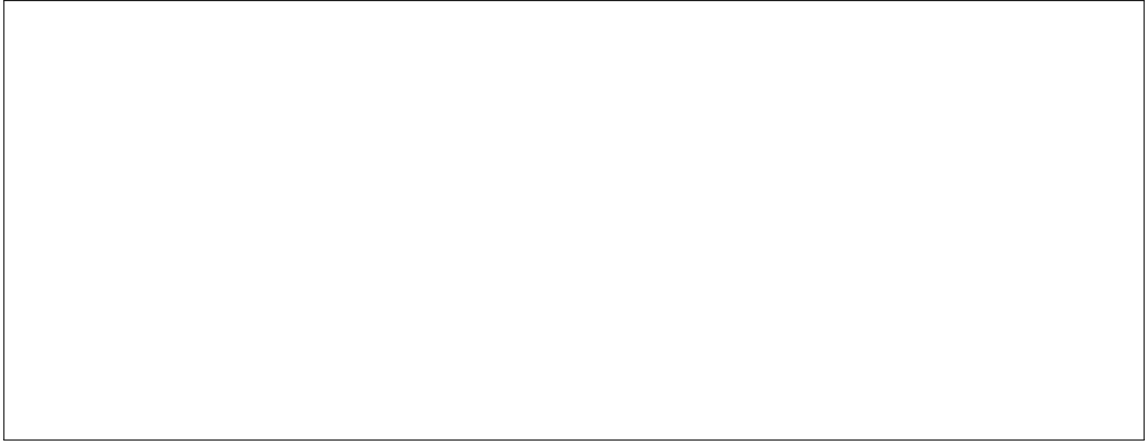


Figure 2: Placeholder: Gross Gain, Cost, and Net Profit by Rollup.

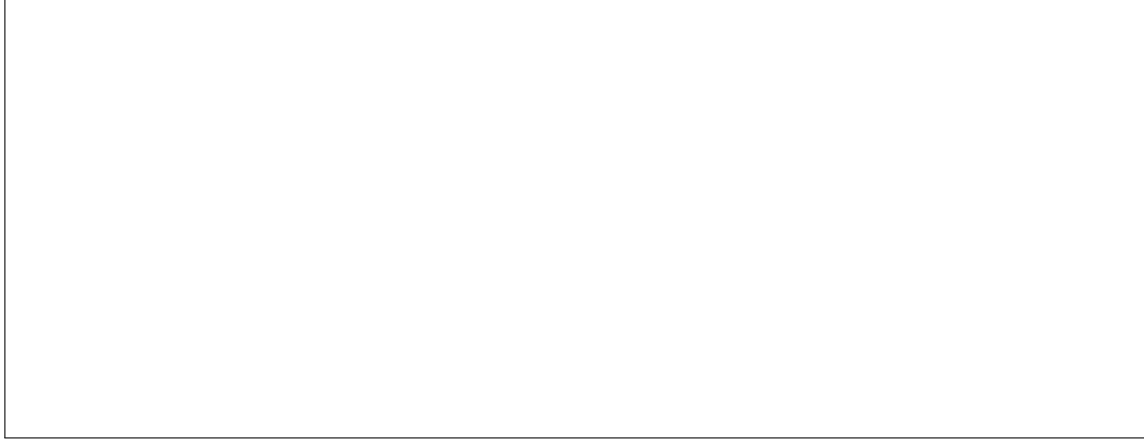


Figure 3: Placeholder: Sensitivity of Simulated Profit to Cross-Chain Time Window.

8 Limitations

Several limitations should be stated plainly at the current stage.

8.1 No bridge-aware execution layer

Cross-rollup paths are created by reserve-token address mapping, not by an explicit bridge object. There is no coded representation of canonical bridge delay, fast-bridge fees, bridge failure probability, or inventory pre-positioning across rollups. Consequently, the cross-chain simulation outputs should be interpreted as state-consistent upper bounds under a 2-second synchronization rule rather than deployable cross-rollup P&L.

8.2 Cross-rollup synchronization remains coarse

Single-chain replay respects block-local update order. Cross-chain replay is necessarily coarser: it uses wall-clock windows and a representative per-pool update within each window. This is a reasonable research approximation, but it is not equivalent to executable cross-domain settlement.

8.3 Provider configuration is only partially reproducible

The current framework verifies the use of chain RPC endpoints for detection and state refresh, but the full provider configuration is not fully documented in the committed outputs. This does not affect the logic of the methodology, but it does matter for exact reproduction of the stored experiments.

8.4 Date parameters are not fully normalized across the pipeline

Most of the stored replay outputs are parameterized around November 1, 2024, but not all data-download defaults are normalized to the same date constants. Reproducing the exact stored ex-

periments therefore requires manual alignment of those parameters.

8.5 Trust Capital remains a future extension

A reliability score under the label *Trust Capital* is conceptually useful, but it is not yet implemented as an explicit computed metric in the current framework. It is therefore more appropriate to present it as a future extension than as an established component of the present methodology.

9 Conclusion

At the current mid-period stage, the project has established a substantive research framework for single-chain realized arbitrage detection and pairwise cross-rollup opportunity replay, built around Uniswap v2/v3 mechanics, Dune and Allium data ingestion, and MongoDB-backed incremental state updates.

The main empirical finding is that the currently stored 2-second cross-rollup replay produces many large theoretical opportunities, whereas single-chain detection outputs are economically smaller but methodologically more credible because they already include transaction-cost accounting. The cross-rollup replay results are therefore valuable as evidence that state fragmentation can generate large apparent opportunities, but they should not yet be interpreted as deployable performance until bridge timing, inventory, and competitive execution are modeled explicitly.

The most important next steps are to strengthen the execution realism rather than simply expand the count of candidate paths. In particular, the project would benefit from a bridge layer, a genuine multi-window sensitivity study, more standardized reproduction settings, and an explicit reliability metric for state quality. With these additions, the framework would move from a strong research prototype toward a more rigorous empirical model of cross-rollup arbitrage under realistic DeFi and MEV constraints.

References

- [1] Hayden Adams, Noah Zinsmeister, and Dan Robinson. *Uniswap v2 Core*. Uniswap, 2020.
- [2] Adams, Zinsmeister, Salem, Keefer, and Robinson. *Uniswap v3 Core*. Uniswap, 2021.
- [3] Optimism Documentation. *Transaction Fees on OP Mainnet*.
- [4] Base Documentation. *Network Fees and OP Stack Fee Components*.
- [5] Dune Documentation. *Analytics API Documentation*.
- [6] Allium Documentation. *Allium Explorer and Query Interface*.